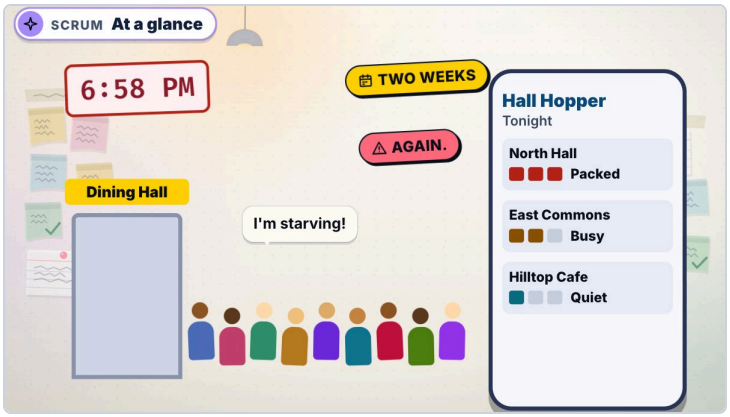
Scrum in one sprint

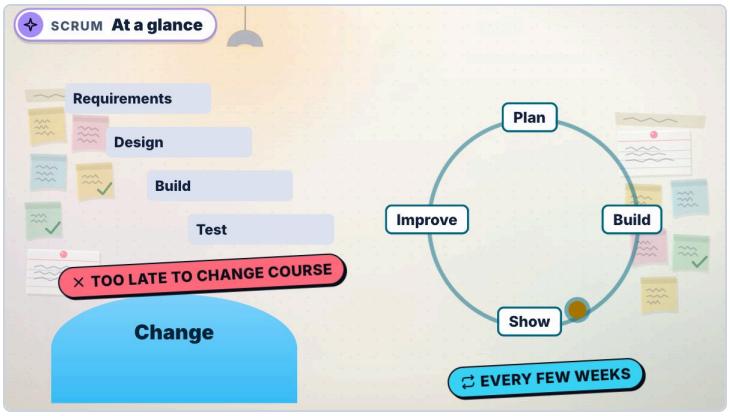
1. Scrum at a glance

1.1 A two-week promise



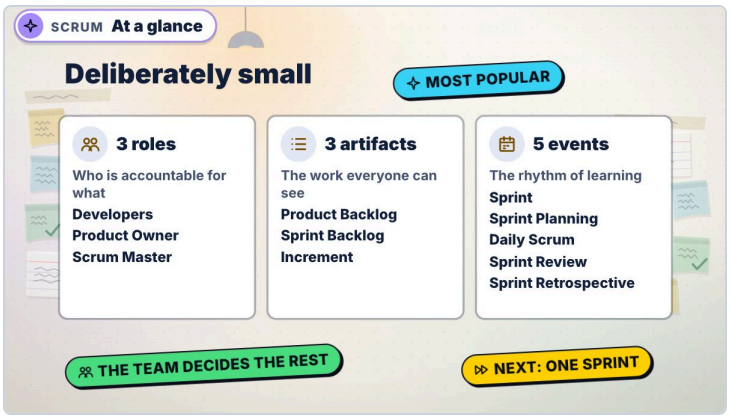
- **Example:** Hall Hopper app: tonight's menus and how packed each dining hall is
- Student team, **two weeks** to ship

1.2 Plan everything, or learn as you go



- Plan everything up front → months of building → too late to change course
- **Scrum:** small working pieces every few weeks; learn from users; adjust

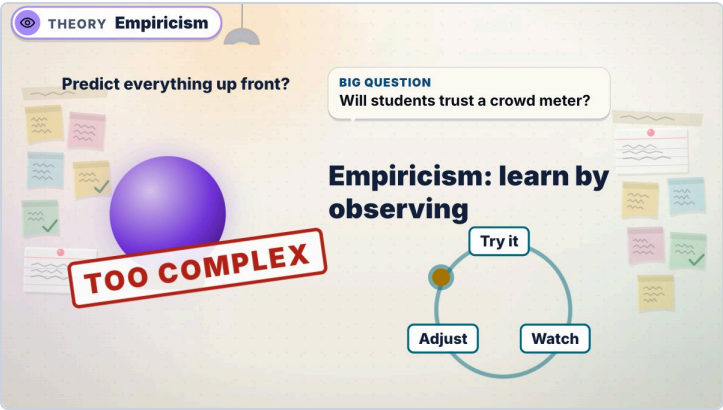
1.3 Three roles, three artifacts, five events



- Most popular agile framework; deliberately small
- **3 roles** (who is accountable), **3 artifacts** (visible work), **5 events** (rhythm)
- The team decides everything else

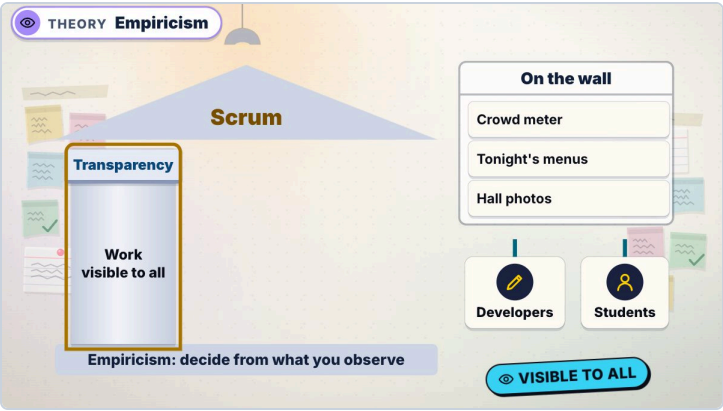
2. Theory: empiricism

2.1 Empiricism



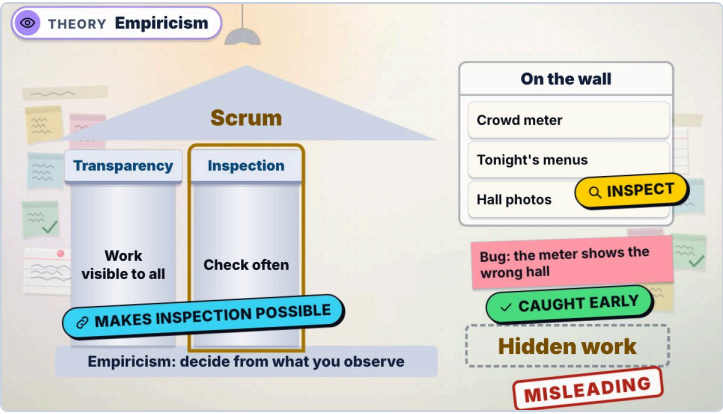
- Software for real people is **complex**: can't predict it all up front
- **Empiricism:** decide from experience and observation

2.2 Transparency



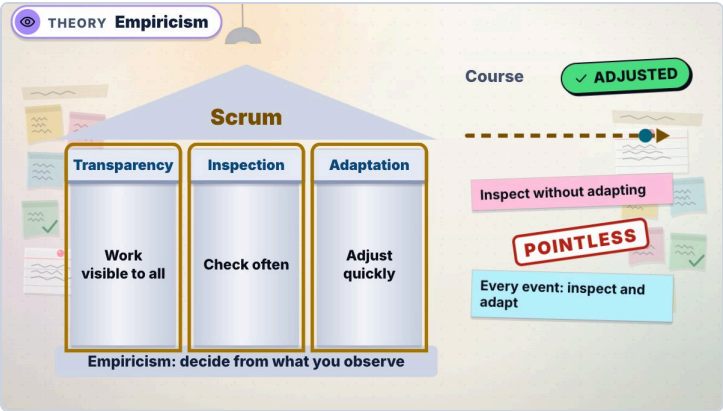
- **Three pillars**; first: **transparency**
- Important facts visible to everyone doing or receiving the work
- Work on the wall, not in heads or private spreadsheets

2.3 Inspection



- **Inspection:** check progress and artifacts often; catch problems small
- Needs transparency: inspecting hidden work misleads

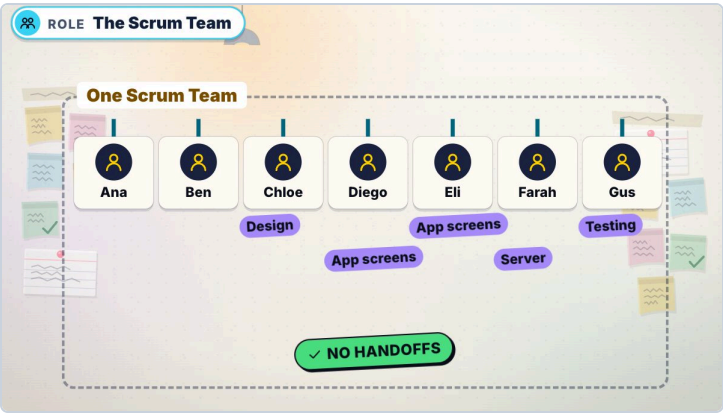
2.4 Adaptation



- **Adaptation:** adjust as soon as work drifts off course
- Inspecting without adapting is pointless
- Every event is a chance to **inspect and adapt**

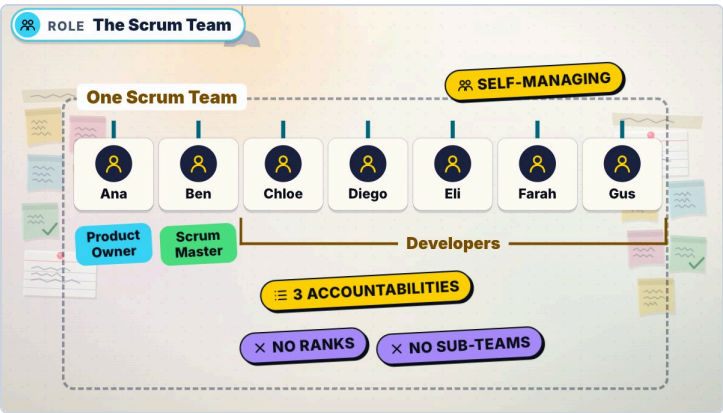
3. Roles: the team and its Developers

3.1 Cross-functional



- **Scrum Team:** 7 people, one product
- **Cross-functional:** every skill needed to ship each sprint; no hand-offs

3.2 Self-managing, with three accountabilities



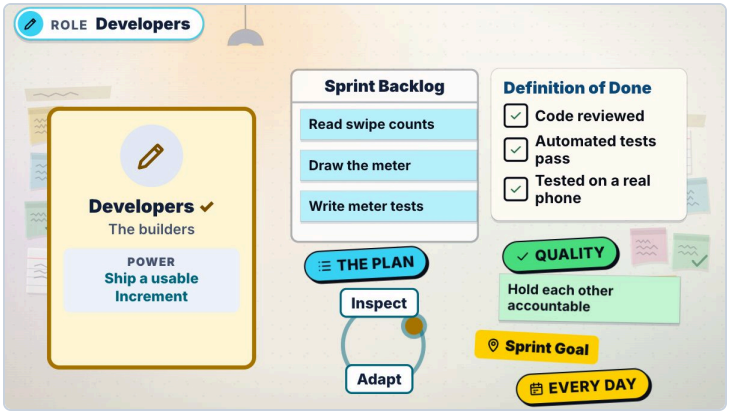
- **Self-managing** (self-organizing): the team decides who, what, when, how
- Three **accountabilities** (roles): Developers, Product Owner, Scrum Master
- No ranks, no sub-teams

3.3 Developers build



- **Developers:** all builders of the usable product, not only coders
- Hall Hopper: designer, tester, 3 programmers

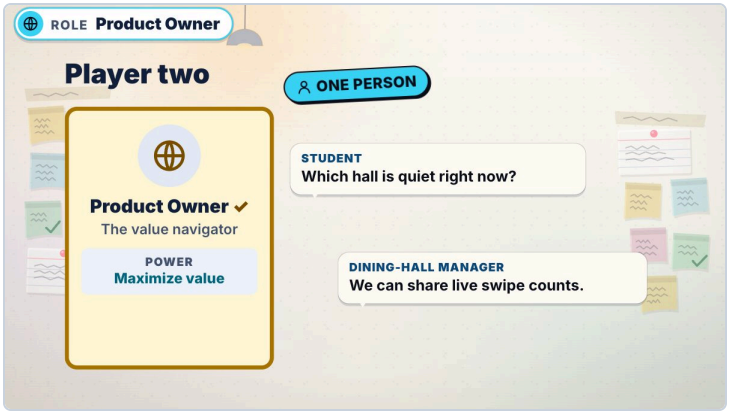
3.4 What Developers own



- Create the **Sprint Backlog** (the sprint's plan)
- Meet the **Definition of Done** (shared quality checklist); hold each other accountable
- Adapt the plan daily toward the Sprint Goal

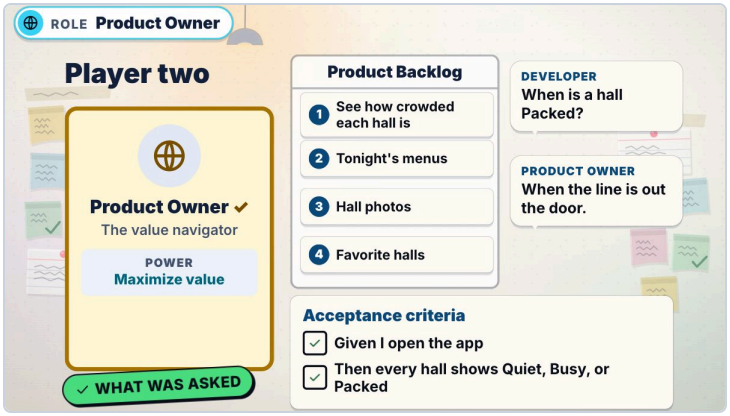
4. Roles: Product Owner

4.1 The value navigator



- **Product Owner:** one person, accountable for maximizing value
- Talks directly with customers to learn their needs and why

4.2 Ordering the work



- Writes **user stories**; **orders the Product Backlog** by value
- Answers Developers' questions; checks **acceptance criteria** on finished work

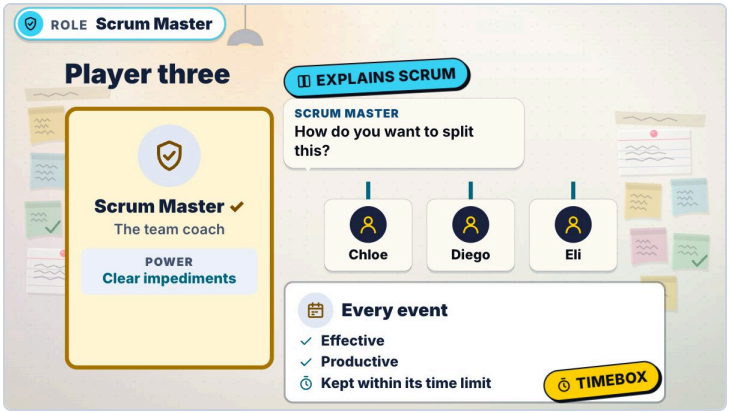
4.3 Three classic traps



- Trap: telephone game → talk with customers directly
- Trap: managers dictate the vision → give real authority
- Trap: little domain knowledge → pick someone who knows users and development

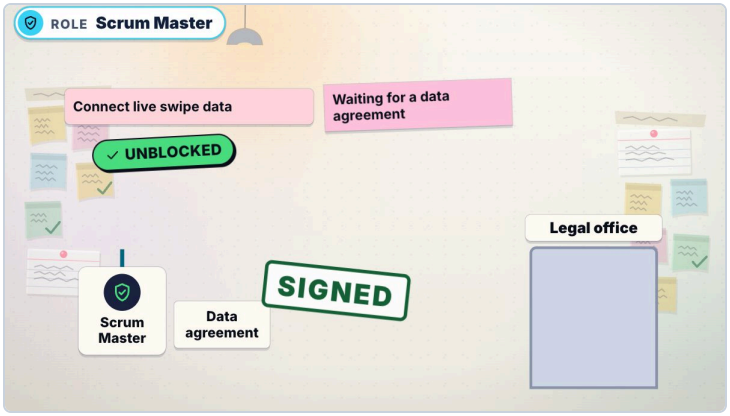
5. Roles: Scrum Master

5.1 The team coach



- **Scrum Master:** team coach; teaches Scrum, coaches self-management
- Keeps events effective, within their **timebox**

5.2 Removing an impediment



- Removes **impediments** the team can't clear alone
- Example: chases the legal sign-off for live dining data

5.3 Not a manager



- Not a product manager (that's the Product Owner) and not a boss
- Serves and shields the team; doesn't command it

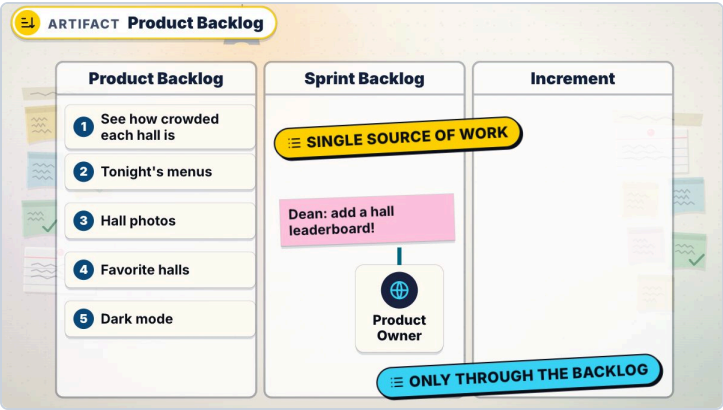
5.4 A healthy tension



- Healthy tension: Scrum Master → **internal quality** (code, process, Developers)
- Product Owner → **external quality** (features, schedule); balance both

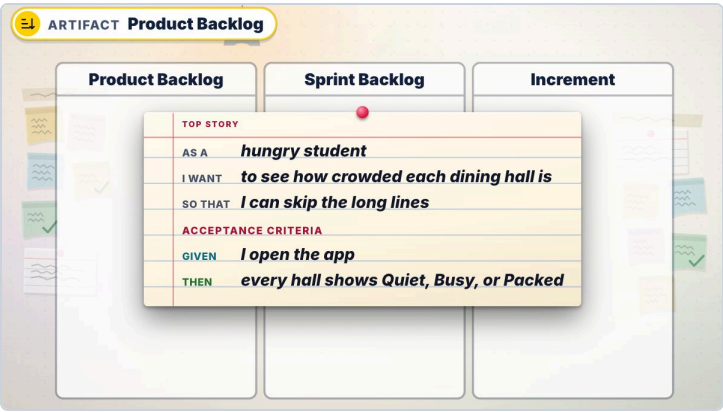
6. Artifacts: Product Backlog

6.1 The single source of work



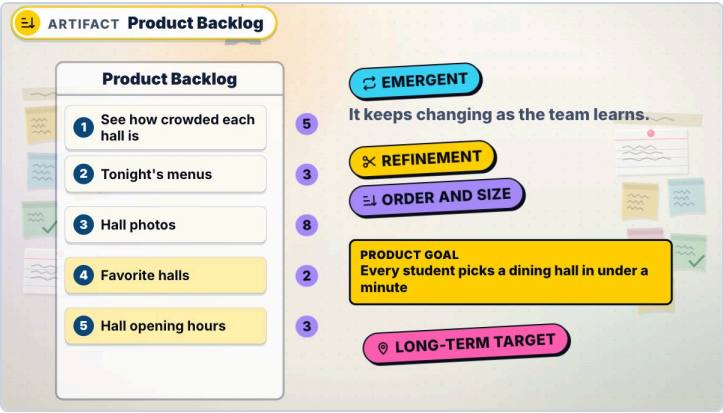
- **Product Backlog:** ordered list of what the product needs
- **Single source of work**: not on it → nobody works on it

6.2 User stories, in order



- Mostly **user stories** with **acceptance criteria**
- Most valuable first: “see how crowded each hall is”

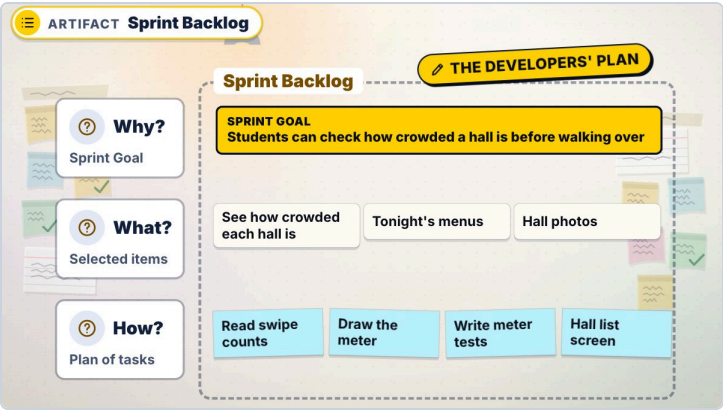
6.3 Emergent, refined, aimed at a goal



- **Emergent:** changes as the team learns
- **Refinement:** split big, fuzzy items; add order and size
- **Product Goal:** the long-term target

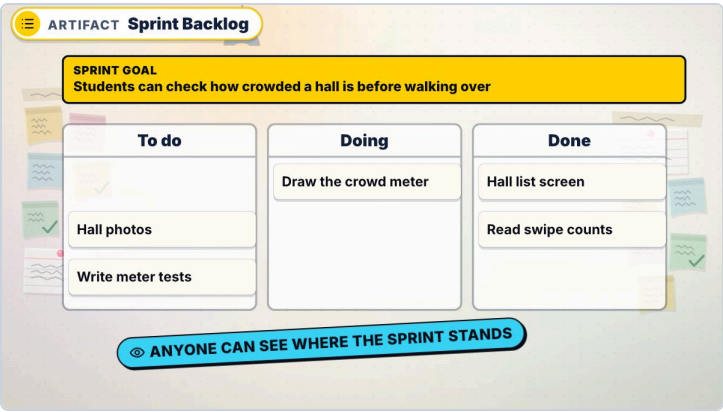
7. Artifacts: Sprint Backlog

7.1 Why, what, and how



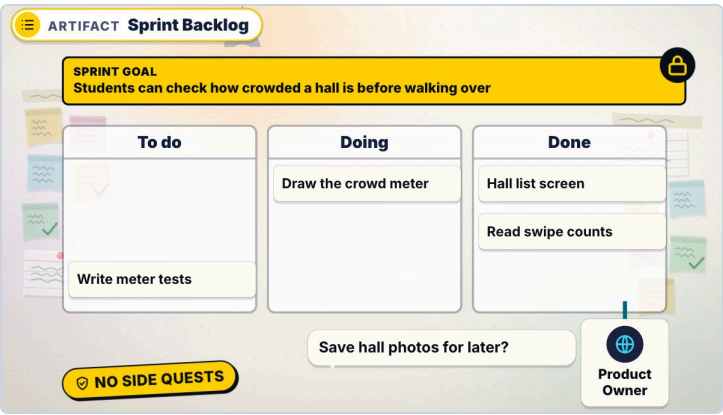
- **Sprint Backlog:** Developers' plan for one sprint
- **Why:** Sprint Goal · **What:** selected items · **How:** tasks

7.2 A plan on the wall



- Lives on the wall, updated in real time by the Developers
- Anyone sees where the sprint stands

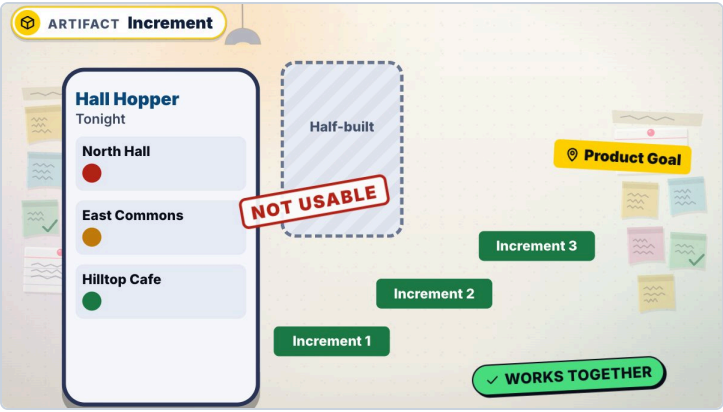
7.3 Scope bends, the goal holds



- Item too big → renegotiate **scope** with the Product Owner
- The **Sprint Goal stays fixed** and keeps the team focused

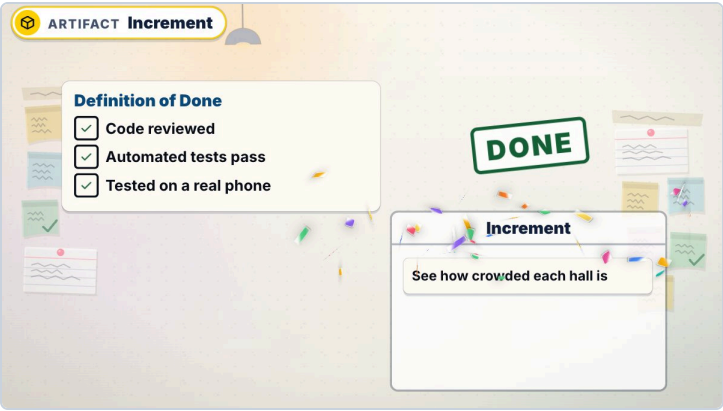
8. Artifacts: Increment

8.1 A usable step



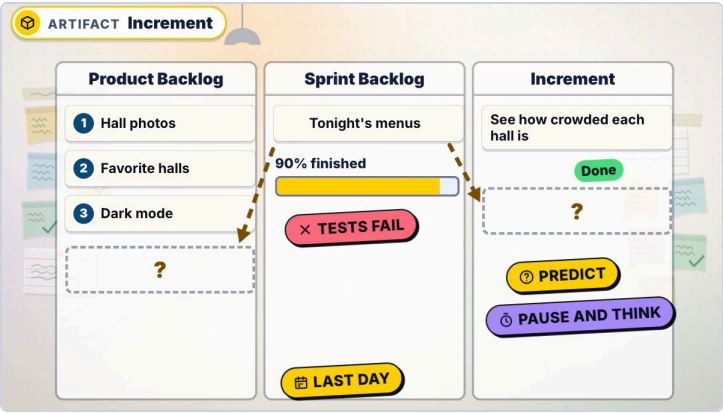
- **Increment:** a usable step toward the Product Goal
- Adds to earlier Increments; verified to work together
- Mockups and half-built screens don't count

8.2 The Definition of Done



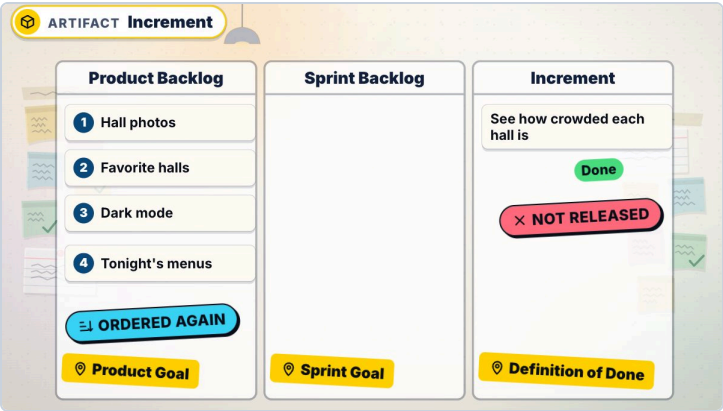
- **Definition of Done:** the shared quality standard
- Example: reviewed, tests pass, tested on a phone
- Meets it → part of the Increment

8.3 Predict



- **Predict:** last day; a story is 90% done, tests failing. Counts? Goes where?

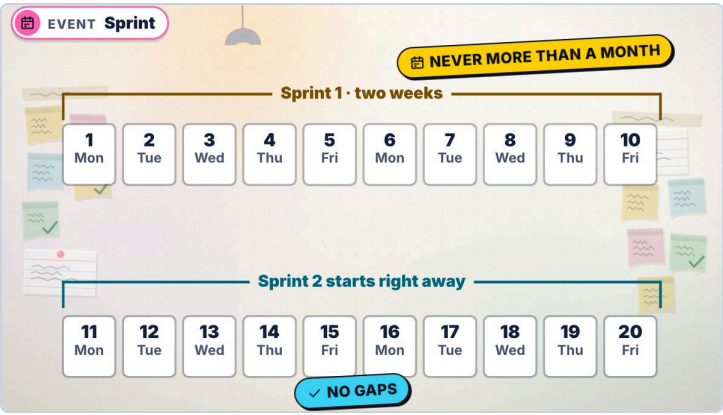
8.4 Unfinished work goes back



- 90% done is not done: not released, not shown as finished
- Back to the **Product Backlog** for reordering
- **Commitments:** Product Goal, Sprint Goal, Definition of Done

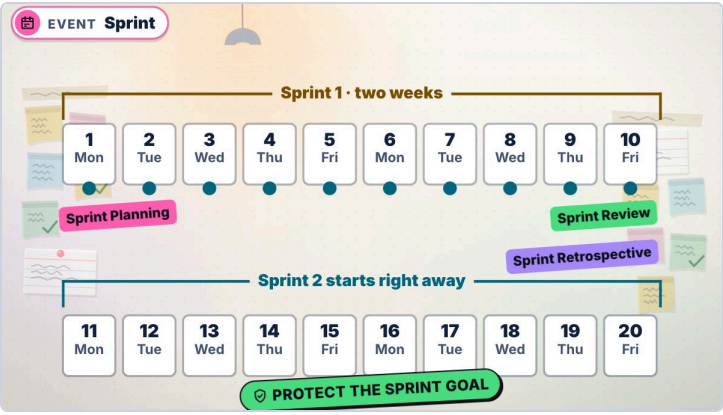
9. Events: the Sprint

9.1 A fixed timebox



- **Sprint:** fixed timebox, usually 2–3 weeks, **never more than a month**
- The next sprint starts immediately; no gaps

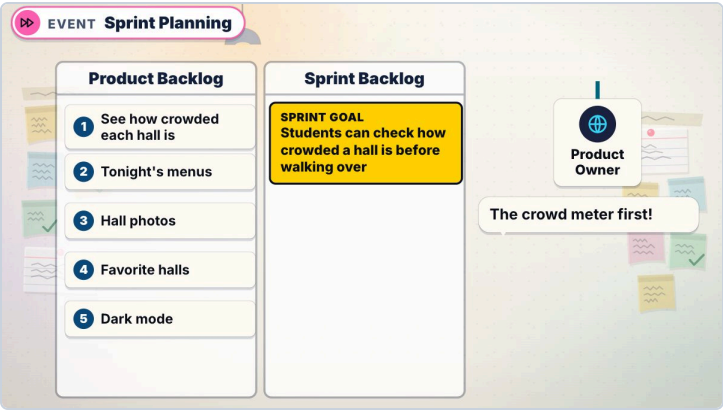
9.2 A container for the other events



- **Container** for the other four events
- Planning → Daily Scrums → Review and Retrospective
- No changes that endanger the Sprint Goal

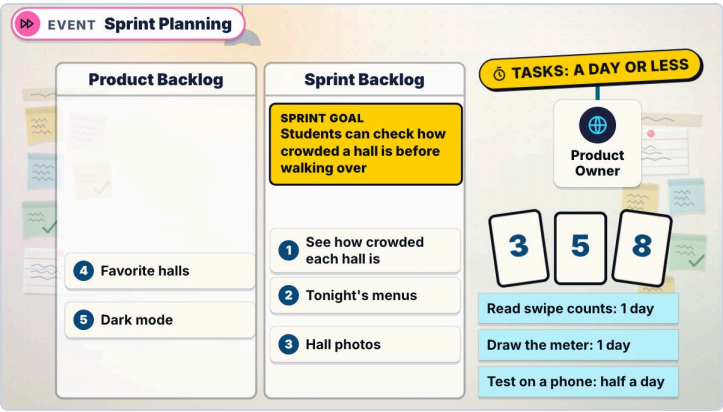
10. Events: Sprint Planning

10.1 Why: the Sprint Goal



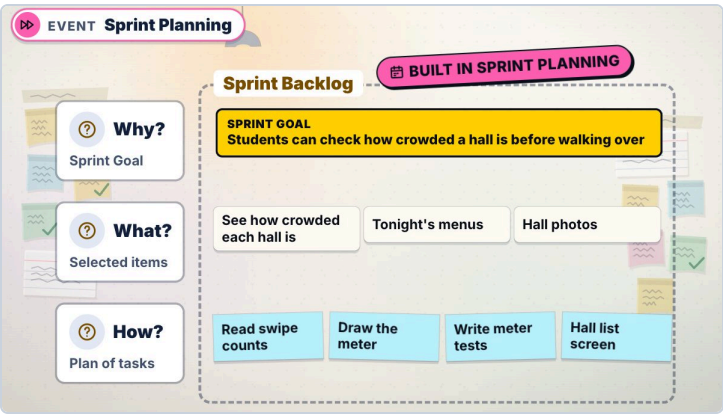
- **Sprint Planning:** the whole Scrum Team starts the sprint
- **Why:** the Product Owner proposes value; the team writes the Sprint Goal

10.2 What, and how



- **What:** Developers pull and size top items (planning poker)
- **How:** tasks of a day or less

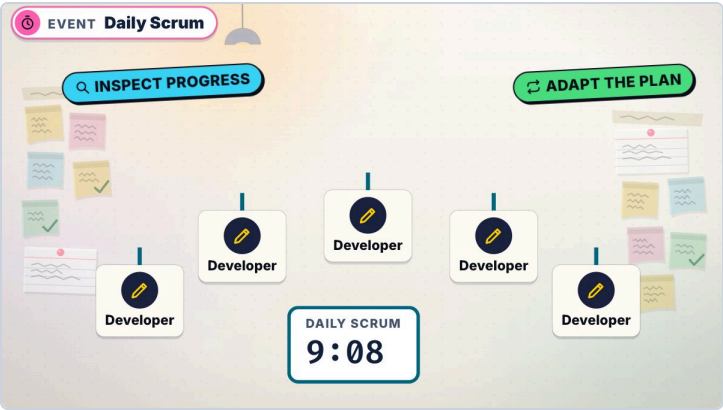
10.3 Three questions, three parts



- Why → goal, what → items, how → tasks: the Sprint Backlog's three parts

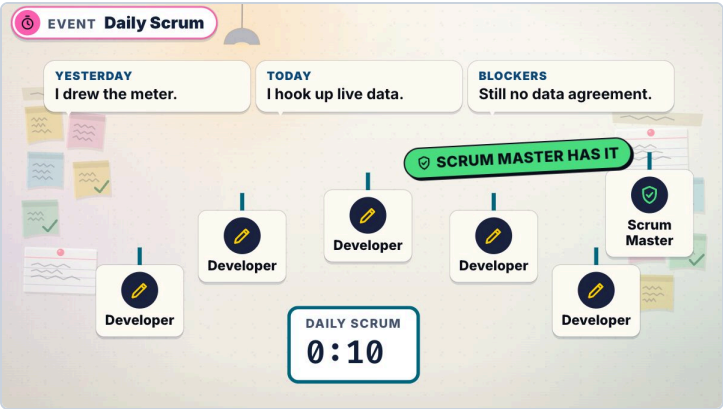
11. Events: Daily Scrum

11.1 Inspect and adapt, daily



- **Daily Scrum** (stand-up): Developers, **15 minutes**, every working day
- Inspect progress toward the Sprint Goal; adapt the plan

11.2 Three questions, one blocker



- Common questions: yesterday? today? blockers?
- The Scrum Master takes the impediment; the meeting moves on

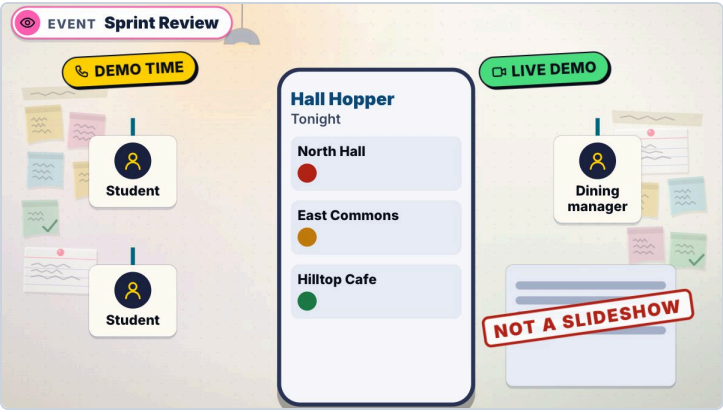
11.3 Not a status report



- **Not a status report** for managers
- Developers coordinate with each other

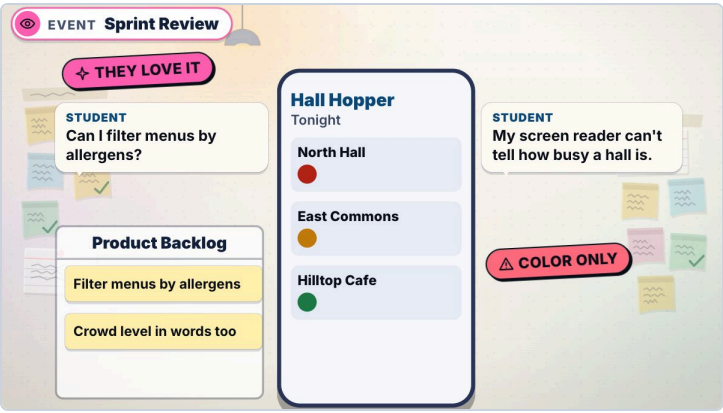
12. Events: Sprint Review

12.1 A working session



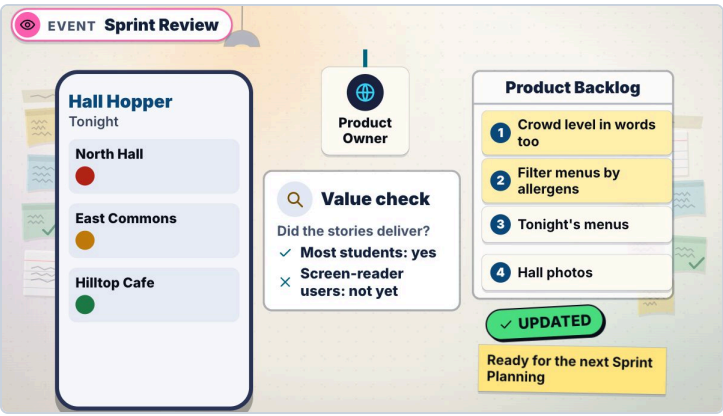
- **Sprint Review:** inspect the Increment with stakeholders
- Progress toward the Product Goal; live demos, not slides

12.2 Feedback changes the backlog



- Feedback: allergen filter; the color-only meter fails screen readers
- New items near the top of the Product Backlog

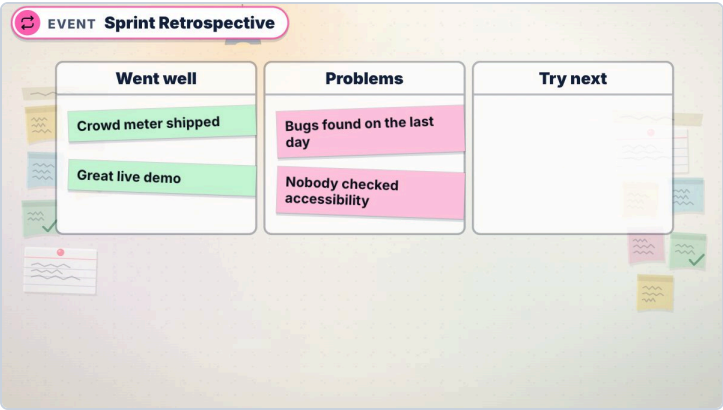
12.3 An updated Product Backlog



- The Product Owner judges the delivered value
- Output: an **updated Product Backlog**

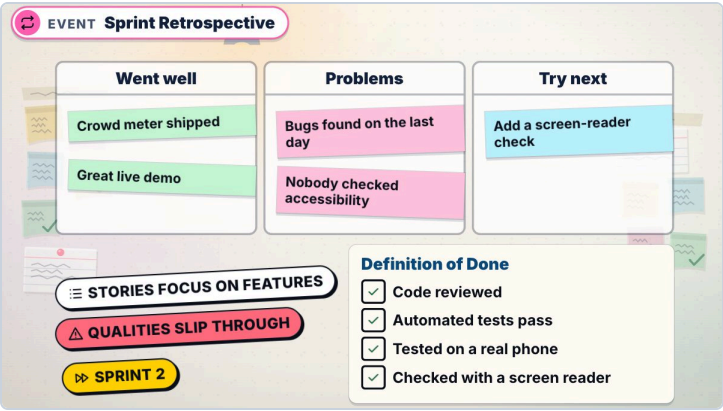
13. Events: Sprint Retrospective

13.1 Inspect the process



- **Sprint Retrospective:** inspects the **process**
- People, interactions, tools, Definition of Done
- What went well? What problems? Solved?

13.2 Change the Definition of Done



- User stories miss qualities like accessibility and performance
- Fix: add a screen-reader check to the **Definition of Done**

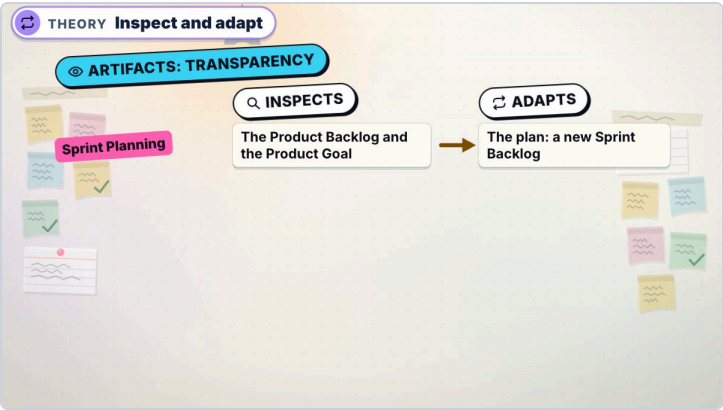
13.3 Product versus process



- Review: **what** to build · Retrospective: **how** to work
- Both feed the next Sprint Planning

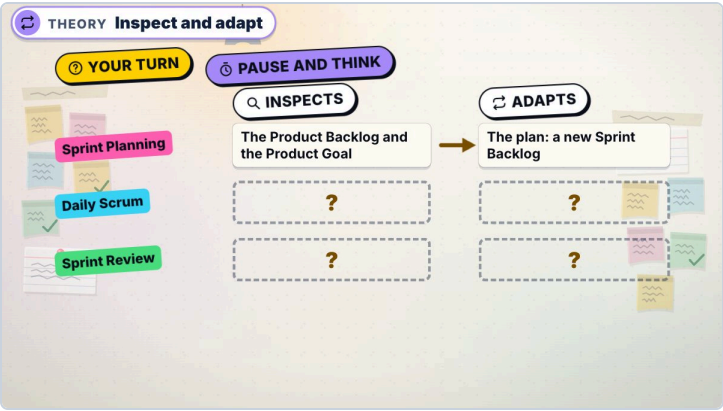
14. Theory: every event inspects and adapts

14.1 Sprint Planning



- Artifacts give transparency; each event inspects and adapts
- **Sprint Planning:** inspects Product Backlog and Goal → creates the Sprint Backlog

14.2 Predict the next rows



- **Predict:** what do the Daily Scrum and the Sprint Review each inspect and adapt?

14.3 Daily Scrum, Review, and Retrospective



- **Daily Scrum:** progress → the day's plan
- **Sprint Review:** Increment → Product Backlog
- **Retrospective:** way of working → process and Definition of Done

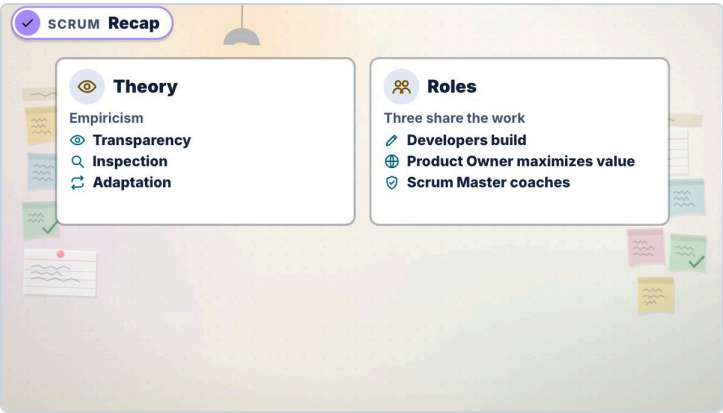
14.4 Learning at three speeds



- The Sprint contains every event
- Learning at **three speeds**: daily, per sprint, across the product

15. Summary

15.1 Theory and roles



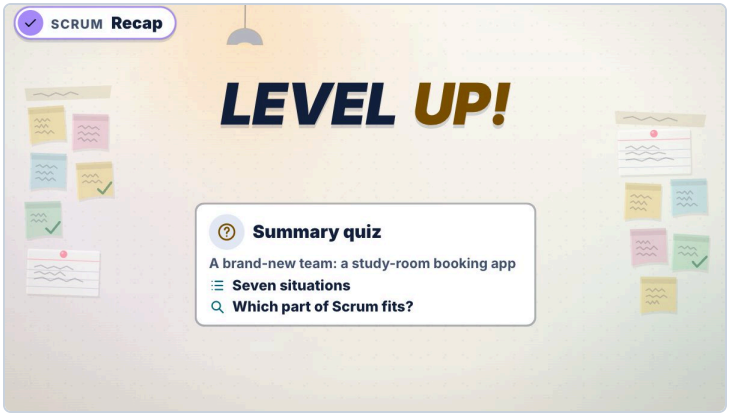
- **Empiricism:** transparency, inspection, adaptation
- Developers build; Product Owner maximizes value; Scrum Master coaches

15.2 Artifacts and events



- Product Backlog → Product Goal; Sprint Backlog → Sprint Goal; Increment → Definition of Done
- Five events: inspect and adapt every sprint

15.3 Try it on a new team



- **Self-test:** the summary quiz on a new team
- Match each situation to a role, artifact, or event